Group Project Milestone 2:

Analytics Implementation — Problem Definition, Model Building and Evaluation

Predicting Delivery Performance and Satisfaction Risk

Two prediction moments on the Olist Brazilian e-commerce marketplace

Group 10

Pan Pinyou	A0329105L
Li Yudan	A0330335R
Zhu Gengyu	A0333773Y

National University of Singapore

IT5006 Fundamentals of Data Analytics

AY2026/2027 Semester 1

Dashboard: `https://it5006-group-project-10.streamlit.app`

Repository:
`https://github.com/gengyuzhu/Team10_IT5006_Ecommerce_Analytics_AY2627Sem1`

1 Problem Definition and Stakeholder Context

Phase 1 left one finding standing above the rest: a late delivery raises the probability of a 1–2 star review from 9.2% to 54.0%, a 5.9-fold increase monotone in how late the parcel is. Phase 2 turns that sentence into a pair of predictions placed at two points in an order's life.

Table 1: The two problems. They share one feature pipeline, one model-family budget and one dataset; what separates them is *when* the model runs.

	Problem 1 — delivery performance	**Problem 2 — satisfaction risk**
Question	How long will this order take, and will it miss its promise?	Will this delivered order draw a 1–2 star review?
Moment	`at_checkout` — the order has just been placed	`at_delivery` — the parcel has arrived, the review is not yet written
Stakeholder	Operations and logistics	Customer experience
Task	**1A regression**: lead time in days, on MAE. **1B classification**: delivered after the promised date, on PR-AUC against its no-skill floor	**Classification**: `review_score` ≤ 2, on PR-AUC against its floor and balanced accuracy
Action	Re-quote the promise, warn the buyer, expedite freight	Reach out before the review lands

The brief permits two problems or one framed twice; we take two, because they are halves of one decision chain. Operations acts before the parcel ships; customer experience acts after it arrives and before the customer writes anything. Problem 1's output is, in substance, Problem 2's most important input.

Problem 1 is the brief's *dual framing*: one underlying quantity, two outputs, so the late-order flag *is* the regression's answer compared against the promised date. Five classifiers trained directly on the binary label run alongside as a control arm, testing whether the flag needs a model of its own; Section 4.2 reports that comparison, including a disagreement between the selection protocol and the forward window that is itself one of this report's findings.

Targets. All three come from the provided tables alone. *Lead time* is `order_delivered_customer_date` − `order_purchase_timestamp` in days, defined for the 97.0% of orders that reached the customer. *Late* is delivery strictly after `order_estimated_delivery_date`, which carries no time component, so lateness is accurate to about a day. *Low review* is `review_score` ≤ 2, defined for the 99.2% carrying a review. Undefined rows are `NaN` and excluded, never imputed to a negative label: the event has not happened rather than gone missing, and a silent customer is not a satisfied one.

Success criteria, registered in advance. Expected ranges for all six headline metrics were written into `config.EXPECTED_BANDS` *before* any Phase 2 model was tuned, from the Phase 1 results and untuned scoping probes (Appendix B). They are the targets the work is judged against and they are a leak alarm — because here the signature of leakage is not a subtle improvement but an implausible one, as Phase 1 showed by adding one forbidden column to the late-delivery model and watching ROC-AUC go from 0.589 to 1.000. A result landing far *above* its band is investigated before it is reported. All six landed inside; none above.

The ceiling deserves stating as plainly as the target: lateness here is volatile rather than trending — 1.4% to 21.4% month to month with Black Friday congestion and the May 2018 truckers' strike — and no order-level feature available at checkout can anticipate a national road blockade. A late-delivery classifier reporting 95% accuracy here would be the majority-class rule or a leak.

2 Data Preparation and Feature Engineering

2.1 From nine tables to one row per order, split three ways

The preparation pipeline is inherited unchanged from Phase 1 (`src/data_load.py`). Three of the nine source tables — items, payments and reviews — are one-to-many against orders, so each is aggregated to the order grain *before* joining; a direct merge yields 118,437 rows rather than 99,441 and silently reweights every mean towards large baskets. The grain is asserted after the join, and `customer_unique_id` is carried throughout because `customer_id` is regenerated per order and is not a person. Restricting to the fully-operational window leaves **99,091** orders. Eleven cleaning decisions with their evidence are recorded as `CLEANING_DECISIONS` in the loader and tabulated in the Phase 1 appendix; `src/report_stats.py` still reproduces every Phase 1 number byte-for-byte after the Phase 2 additions, which `tests/test_leakage.py` asserts.

Each period has exactly one job. **Train** (2017-01-01 – 2018-04-30, 73,008 orders) fits every model, with expanding-window cross-validation inside it tuning hyperparameters. **Validation** (2018-05-01 – 2018-06-30, 13,156) sets every decision threshold and fits the margin-to-risk calibration. **Test** (2018-07-01 – 2018-08-31, 12,927) is scored once, at the end, for models already chosen. The split is chronological rather than stratified-random because the target drifts, and additionally disjoint on the customer: 2,997 people ordered more than once, so a purely temporal cut can still put one of a person's orders on each side, and the **227** straddling buyers are removed from a clean test subset used in Section 4.3.

2.2 Features, and the moment each becomes knowable

Phase 1's 25 features carry forward unchanged: 6 calendar (month, weekday, hour, weekend, business hours, the Black Friday window), 11 basket-and-money (`log_price`, `freight_ratio`, `log_weight`, volume, item and seller counts, instalments), 4 geographic (`log_distance`, `same_state`, customer state, seller region), 2 for the promise and 2 catalogue. The design carries no exact duplicates — Phase 1's first draft held `total_price` alongside `log_price` and `total_payment`, which makes a linear baseline meaningless — and `purchase_year` is absent because the split is chronological, so a linear term on it would extrapolate into an unseen year. Phase 2 adds **eight track-record features**, taking the checkout set to 33 and the at-delivery set to 36: a seller's mean lead time, late rate, low-review rate and order count; a route's mean lead time; a category's low-review rate; and the platform's own lead time and late rate. The at-delivery regime additionally sees three columns describing how delivery actually went.

2.3 Track records, and the trap inside them

A seller's history is the strongest thing a marketplace knows about an order at checkout, and the easiest place here to leak, because of a distinction that is easy to miss: *past* means the earlier order had **closed** before this one was placed — not merely that it was placed earlier. A parcel bought last week that has not yet arrived has no lead time yet, so ordering the history by *purchase* time would let every order average in outcomes that had not happened. That is Kaufman et al.'s *illegitimate feature* [7] in a convincing disguise, and it would be invisible in the metrics because it would simply look like a better model. Every aggregate is therefore an as-of join against the moment the earlier outcome became observable — `order_delivered_customer_date` for delivery, `review_creation_date` for reviews — with a strict boundary.

Two further guards. A mean over one previous order is noise rather than a rate, so below a floor of five prior orders the value is undefined and a fallback chain takes over (seller → route → seller state → platform), after which 0.95% of orders lack any history value. And cold start is information rather than error: `hist_seller_n` carries how many past orders the estimate rests

on, so a model can discount a seller it has barely seen. The platform features additionally use a **28-day trailing window** rather than an expanding mean — lead time halves across the analysis window, so an average over all history reports roughly 13 days while the current month runs at 8 — and their range, 5.2 to 17.1 days, is a direct measure of the drift the models must survive.

Because a leak here would be invisible in the metrics, the construction is checked rather than reviewed: `tests/test_leakage.py` rebuilds `hist_seller_lead_days` from first principles for 200 random orders and requires exact agreement, and a second check confirms the platform tracker *lags* rather than anticipates (Appendix A).

2.4 Leakage: one rule per prediction moment

Leakage here is not one rule but one per *prediction moment*, because what is known depends on when the model runs. *At checkout* nothing about fulfilment or the customer's opinion exists, so delivery dates and everything derived from them, the review score and its timestamps, `order_approved_at`, `shipping_limit_date` and `order_status` are forbidden. *At delivery* only the review and its timestamps remain forbidden — the delivery outcome is now history, and the brief itself lists "delivery performance" among the features for that problem. `assert_no_leakage(features, regime)` enforces the correct set before any model is fitted, is tested in both directions, and raises on an unrecognised regime name, because passing the wrong regime is itself the mistake it exists to catch.

`order_estimated_delivery_date` is admissible at checkout and matters: the customer is shown that promise when the order is placed, so it precedes the outcome and encodes whatever Olist's own routing knew. It is a useful feature and — Section 4.1 — a poor prediction. The cost of getting the regime wrong is measurable: adding one forbidden column, the realised delivery time, lifts the late-delivery model's ROC-AUC from 0.589 to **1.000**. That is not a result; it is the target restated.

3 Modelling Methodology

3.1 Model family budget and pipelines

The brief allows two to three families and asks for depth rather than breadth. We use three, reused across both task types, each arranged as a *complexity ladder* whose rung 0 is the family's simplest variant — which is the only way "the added complexity was worth it" becomes a finding rather than an assumption. **A — Linear** runs `LinearRegression` / `LogisticRegression` at rung 0 and `Ridge` / ℓ_2-penalised logistic at rung 1. **B — Tree-based** runs `DecisionTree` → `RandomForest` → `HistGradientBoosting`. **C — Ensemble** is soft `Voting` and `Stacking` assembled from the winners of A and B; its members are complete pipelines rather than bare estimators, because the two families need different encodings and one shared preprocessor would force a single choice on both. The full specification, search spaces and selected values are in Appendix C.

Every model is a `scikit-learn Pipeline`: median imputation, one-hot encoding of the four categoricals with rare levels folded into one bucket, then — for the linear family only — standardisation, since trees are invariant to monotone rescaling and a shared penalty is not [10]. The linear family additionally drops one level per categorical, which is a correction rather than a preference: Phase 1 reported that ordinary least squares was unusable here, "rank-deficient, so its coefficients explode", and re-running it four ways (Appendix D) reproduces the observation and corrects the conclusion. The deficiency is exactly four, one per categorical; least squares still returns a usable *prediction* (test MAE 4.181, indistinguishable from Ridge) but a coefficient vector of order 10^{13} whose cancellation fails on one of five folds — something a single train/test

split would never have shown. Dropping a level restores full rank, and once it does, Ridge and OLS agree to three decimals. Regularisation buys numerical stability here, not accuracy.

3.2 Cross-validation, and how a model gets chosen

Delivery performance drifts hard — mean lead time falls **5.34 days** between training and test — so shuffled k-fold lets a model learn from the future, the invalidity Bergmeir and Benítez [1] formalise. Hyperparameters are tuned by `TimeSeriesSplit` with five expanding windows inside the training period, searched with `RandomizedSearchCV` [2] over 30 draws per model — six for the random forests, which cost one to two orders of magnitude more per fit — scoring `neg_mean_absolute_error` for regression and `average_precision` for classification.

Selecting between tuned models is a separate question, and both obvious answers fail here for reasons that are properties of the data. Averaging cross-validated scores fails because the folds differ enormously in difficulty — lead time runs 12–17 days across the 2017 folds against 8.3 in the test window — so every model's CV error sits about a day above its forward error and the *ordering* does not survive: cross-validated MAE ranks Ridge above the boosted trees, while every later window and the test period rank them the other way round. Using the validation period alone fails too: two months is short, and these two pull in opposite directions, the late rate being 8.2% in May 2018 and **1.4%** in June, the lowest in the series.

Each candidate is therefore scored on all six out-of-sample windows — the five expanding folds and the validation period — **ranked within each**, and the winner is the best mean rank over the **three most recent**. Ranks rather than scores, because ranks are the only thing comparable across windows of different difficulty and different base rates; three windows because that covers roughly February to June 2018, recent enough to resemble deployment and wide enough that neither of the validation period's two odd months decides alone. All six are reported in `report/results/selection_*.csv`. A shuffled, customer-disjoint `StratifiedGroupKFold` is computed alongside as a contrast exhibit and never used to choose anything. The selected models are *not* refitted on train plus validation before the test window is scored: a production system should, but that would decouple the shipped artifact from the threshold and calibration fitted against it, and what is reported here is exactly what is deployed.

3.3 Imbalance, and reproducibility

Both targets are imbalanced — 7.4% late and 11.0% low review in the test period — so accuracy alone is meaningless and every results row carries `majority_accuracy` beside `accuracy`, with PR-AUC reported against its no-skill floor [13]. Phase 1's literature review argued from van den Goorbergh et al. [15] that correcting imbalance does not improve discrimination and badly miscalibrates the result, and committed Phase 2 to testing rather than assuming it (Section 4.3). We fit on the observed prior and move the *decision threshold* instead, tuned on the validation period by maximising F1 — which changes the decision without distorting the probability. Calibrated probabilities are a requirement rather than a nicety, because Section 5.2 multiplies them by a cost [5]; they are reported, not assumed [8]. No resampling is used anywhere.

`SEED = 42` throughout. `src/train_models.py` fits everything and writes the models, metadata and every result table; `src/phase2_stats.py` distils those into `report/stats_phase2.txt`, and no number enters this report that it cannot regenerate. Every figure is drawn from those same tables, so a figure cannot disagree with one. Artifacts record the `scikit-learn` version that wrote them and the loader checks it; `tests/test_leakage.py` runs 32 validation checks (Appendix A).

4 Results

Everything below is the untouched test window, 2018-07-01 to 2018-08-31, scored once for models already chosen. Cross-validated and validation columns sit beside it because the disagreement between them is part of the result. Figures are in Appendix H.

4.1 Problem 1A — lead time

Table 2: Lead-time regression: two reference points that are not models, then each family's complexity ladder. **Bold** is the selected model. Figure 1.

Model	Family	CV MAE	Val MAE	Test MAE	RMSE	R^2
Mean of the training period	baseline	—	—	6.645	7.572	−0.990
Olist's own delivery promise	baseline	—	—	9.788	12.155	−4.130
LinearRegression *(rung 0)*	A linear	5.242	4.965	4.181	5.639	−0.104
Ridge *(rung 1)*	A linear	5.201	5.164	4.309	5.775	−0.158
DecisionTree *(rung 0)*	B tree	5.539	5.927	6.116	8.664	−1.606
RandomForest *(rung 1)*	B tree	5.457	5.513	4.956	6.169	−0.321
HistGradientBoosting *(rung 2)*	B tree	5.495	4.802	4.141	5.517	−0.057
VotingRegressor (A+B)	C ensemble	5.315	**4.784**	**3.898**	**5.342**	+0.009
StackingRegressor (A+B)	C ensemble	5.443	4.792	4.121	5.430	−0.024

The selected model puts mean absolute error at **3.90 days**: **41.3% below** a constant fitted on the training period and **60.2% below the platform's own promised date**. That second comparison is the one operations cares about, and its direction is not anomalous — a promised date is a commercial decision, padded because a broken promise costs returns and retention [3]. It is an excellent feature and a poor forecast.

Three things matter more than the winner. **Regularisation does not pay:** Ridge is *worse* than plain least squares on validation (5.164 against 4.965) and on test (4.309 against 4.181), with a tuned α of 660 that cross-validation preferred and no later window did — within family A, rung 1 fails to beat rung 0. **Inside family B complexity does pay**, monotonically on validation: 5.927 → 5.513 → 4.802. And **only the ensemble has a positive R^2**; every other model is negative, which is the signature of a level shift rather than of a useless model, and the reason MAE leads throughout. The blend beats both its members (4.181 and 4.141 against 3.898) by a specific mechanism: a tree cannot predict outside the range of the targets it was fitted on, because every leaf averages training values, and test-window lead times are systematically shorter than anything common in training. A linear model has no such bound and extrapolates the level (Figure 2). Averaging is one member covering the other's exact failure, which is why the weights are fixed at 1:2 in the tree's favour rather than tuned.

4.2 Problem 1B — late against the promise

Table 3: The late-order flag, best of each route; the full seven-candidate table is in `report/results/clf_p1b.csv`. No-skill PR-AUC floor 0.074. Figure 4.

Route	Model	ROC-AUC	PR-AUC	lift	bal. acc	F1
direct	best of five (HistGradientBoosting)	0.584	0.092	1.24×	0.524	0.134
derived	predicted days > promised days, unfitted	0.569	0.109	1.47×	0.569	0.214
derived	**regression margin, calibrated**	**0.782**	**0.214**	**2.89×**	**0.705**	**0.235**

The deliverable is the derived route, and that follows from the framing fixed in Section 1 rather than from the numbers: Problem 1 is the brief's dual framing, so the flag *is* the regression's

own answer compared against the promised date. The five direct classifiers answer a different question — does the flag need a model of its own? It does not. The derived route reaches **2.89**× the no-skill floor where the best direct classifier reaches 1.24×, and it costs no additional model: the margin $\hat{y}$ − promised is mapped to a probability by a single-parameter logistic [11] fitted on validation. Even the zero-parameter rule — flag whenever the model expects a miss — beats every fitted classifier.

The selection protocol got this one wrong, and that is worth reporting. Applied to these seven candidates the rolling-origin rule of Section 3.2 ranks a direct classifier first. On every in-training window the candidates sit within noise of one another — in the fifth fold they span 0.206 to 0.362, with the derived route mid-pack — so the rule has almost nothing to separate them on, and separates them on noise; on the forward window the gap opens to a factor of 2.3. The monthly series explains why: the later training folds and the validation period are dominated by the February–April 2018 congestion, when the late rate ran at 16.0% and 21.4%, and by the May strike. A classifier can fit a regime; the derived route encodes a mechanism, and mechanisms transfer. We report the disagreement rather than presenting the rule as having chosen this, and note that confirming the preference would need a further forward window we do not have.

4.3 Problem 2 — low-review risk

Table 4: Low-review classification at delivery. Accuracy is printed beside the majority-class rule that scores 0.890 while catching nothing. No-skill PR-AUC floor 0.110. Figure 5.

Model	Fam.	Val PR	ROC	PR-AUC	lift	bal. acc	F1	acc	Brier
LogisticRegression *(r0)*	A	0.320	0.659	0.291	2.63×	0.636	0.345	0.847	0.090
LogisticRegression (ℓ_2) *(r1)*	A	0.320	0.659	0.291	2.63×	0.636	0.342	0.843	0.090
DecisionTree *(r0)*	B	0.389	0.722	0.396	3.59×	0.677	0.461	0.899	0.080
RandomForest *(r1)*	B	0.403	0.744	0.405	3.67×	0.680	0.470	0.901	0.078
HistGradientBoosting *(r2)*	B	0.407	0.749	0.415	3.76×	0.680	0.471	0.902	0.078
VotingClassifier (A+B)	C	0.404	0.747	0.407	3.69×	0.683	0.448	0.885	0.079
StackingClassifier (A+B)	C	**0.409**	0.749	0.414	3.75×	0.681	0.470	0.901	0.078

The selected model reaches **ROC-AUC 0.749**, **balanced accuracy 0.681** and **PR-AUC 0.414 — 3.75× the no-skill floor**. Its raw accuracy is 0.901 against a majority-class rule scoring 0.890, which is exactly why accuracy is printed but never led with. The tree family beats the linear one by 1.43× on PR-AUC — but a *single depth-limited decision tree* already reaches 0.396 of the eventual 0.414, so 96% of the achievable signal comes from one interpretable tree. And the model does not memorise buyers: on the 12,617 test orders from customers never seen in training it scores PR-AUC **0.415** against 0.414 overall.

Class weighting: the ablation Phase 1 promised. On the best tree model, weighting the loss moves ROC-AUC from 0.749 to 0.747 and PR-AUC from 0.415 to 0.411 — no gain — while the Brier score degrades from **0.0778 to 0.1331**, a 71% deterioration, and mean predicted risk inflates from 0.119 to **0.337** against an actual rate of 0.110. The weighted model overstates every customer's risk threefold (Figure 6). We therefore fit on the observed prior and move the threshold instead. Tuned on validation by maximising F1 it lands at 0.19 rather than 0.50, which on the test window raises F1 from 0.414 to **0.470** and recall from 0.297 to 0.398 at a precision cost of 0.686 to 0.575, flagging 7.6% of delivered orders instead of 4.8%. Calibration holds where the volume is: in the two bins containing 11,892 of the 12,855 test orders the gap between predicted risk and observed rate is under 0.013.

4.4 What the cross-validation protocol was telling us

Table 5: One model, three protocols. Raw average precision disagrees; the same numbers divided by each period's no-skill floor do not. Figure 8.

Protocol	PR-AUC	fold spread	no-skill floor	lift
`TimeSeriesSplit`, expanding window	0.562	±0.091	0.158	3.55×
`StratifiedGroupKFold`, shuffled	0.580	±0.006	0.158	3.66×
Forward test window — the truth	0.414	—	0.110	3.75×

Both cross-validated estimates sit about 0.16 above the forward result, which looks like catastrophic optimism — and almost all of it is mechanical. Average precision's no-skill baseline *is* the positive rate [13], and the low-review rate falls from 15.8% inside the training period to 11.0% in the test window; normalised by that floor the three agree to within 0.2 of a multiple. **Raw PR-AUC is not comparable across periods with different base rates; its lift over the floor is** — and quoting the raw number alone would have supported either a claim of severe overfitting or a claim of none. The shuffled protocol adds a second warning: its fold-to-fold spread is ±0.006 against the temporal protocol's ±0.091, an interval fifteen times narrower on an estimate no closer to the truth.

4.5 Does the third family earn its place, and where do the models fail?

Family C is selected on both problems and the honest answer differs. On **lead time** it earns its place: the Voting blend improves on the better of its members by 0.24 days of MAE (5.9%) and is the only model in Table 2 with a positive R^2, for the extrapolation reason above. On **review risk** it does not: Stacking wins the rank vote by a nose, then matches the boosted trees to three decimals on ROC-AUC and comes in 0.001 *below* them on PR-AUC, so a deployment could ship the single boosted model with no measurable loss. That is worth saying because the ensemble was *flattered* here: stacking builds meta-features with `cross_val_predict`, which needs the inner folds to partition the training set, and an expanding window partitions nothing — so the inner CV must be contiguous `KFold` and the meta-learner sees predictions partly made by models trained on later orders (Appendix C). Being flattered and still failing to separate is a safer conclusion than the reverse.

Across geography the review model is remarkably stable: ROC-AUC runs 0.744 to 0.778 across the five Brazilian regions, with lift over each region's own floor between 3.6× and 5.0×. The regression is not (Figure 10): MAE runs from 3.42 days in the Southeast, where most orders are, to **6.84 days in the North**, and the bias is positive everywhere — +1.15 days in the Southeast rising to +4.77 in the North. The model *over-predicts lead time everywhere*, most severely where deliveries are longest, which is exactly what a model fitted on a slower past does when fulfilment has improved underneath it, and the strongest argument here for refitting on a rolling basis.

5 Interpretation and Business Insight

5.1 What the models rely on

Importance is measured by permutation on the held-out window rather than by impurity on the training set: impurity-based scores are computed where the model was fitted and inflate high-cardinality features [14], which with 74 product categories in play is not hypothetical (Figure 9).

Problem 2 has one dominant driver, and it is actionable. How late the parcel was carries 0.206 of PR-AUC on its own, how long it took a further 0.074, the size of the order 0.045;

nothing else clears 0.011. The top two account for **76.9%** of total positive importance. That is the honest description — the model asks how late the parcel was and how big the order was — and it explains why a single depth-limited tree captured 96% of the achievable signal. For the customer-experience team that is good news: the driver is something the business already measures and can already act on.

Problem 1A's importances must be read with care. No single feature carries much — the largest, `lead_payment_type`, costs 0.034 days of MAE, and the top two together only 30% of total positive importance — and that is a property of the measurement. Permutation importance systematically understates correlated predictors, because permuting one leaves its substitutes intact, and `estimated_days`, `hist_route_lead_days`, `hist_seller_lead_days` and `log_distance` all encode overlapping information about how far and how long; the selected model is additionally a blend, so permuting a column degrades only part of it. The reading that survives is negative and worth having: this model has no single point of failure.

5.2 From a probability to a decision — and when not to act

A probability is not a decision. Turning one into the other needs a cost, and Olist publishes none — so what follows states its assumptions openly and treats the conclusions as shapes rather than numbers [5]. The arithmetic is one line: acting is worth it only where precision $> c_{\text{action}}/c_{\text{miss}}$.

Table 6: The decision layer under stated (not measured) costs. Break-even precision is what decides whether a flag is worth acting on at all.

Flag	c_{action} : c_{miss}	break-even	best achieved	verdict
Low-review risk, at delivery	R\$5 : R\$40	0.125	0.37 at the optimum	**act**
Late-order flag, at checkout	R\$12 : R\$40	0.300	0.29 at any volume	do not expedite

The review flag pays, comfortably. Contacting a customer who was never going to complain costs perhaps R\$5 of agent time; a 1–2 star review that could have been headed off costs perhaps R\$40 — standing in for the damage Deshpande and Pendem [4] trace through to seller sales. Break-even precision is 0.125 and the model reaches 0.369 at its expected-benefit optimum, flagging 1,927 of 12,855 delivered orders (15.0%) of which 712 do go on to leave a low review, for a net **R\$1,466 per 1,000 delivered orders** (Figure 7). The margin is wide enough that the conclusion survives a wrong cost estimate.

The late-order flag does not pay — for an expensive action. This is the more useful finding, and reporting discrimination alone would have missed it. Expediting freight costs perhaps R\$12 against the same R\$40 of downstream damage, so break-even precision is 0.300 — and the derived flag never reaches it: precision plateaus at **0.29** and falls as volume rises, so expected net benefit is negative at every threshold flagging a meaningful number of orders. A model with ROC-AUC 0.782 and 2.9× the no-skill floor is genuinely informative and still not worth paying R\$12 an order to act on. That changes *which* action it justifies rather than making it useless, because break-even falls with the cost of the intervention and two of the available interventions cost almost nothing. **Re-quoting** the promised date is free: the platform's own estimate is off by 9.79 days of MAE against the model's 3.90, so the date shown at checkout could be materially tightened, which Cui et al. [3] show trades directly against returns and retention. **Pre-warning** the buyer is nearly free, and at a threshold of 0.11 the flag catches **82.6%** of the orders that will miss their date.

5.3 What each stakeholder does with this

Operations, at checkout: tighten the quoted delivery date using the model's estimate, and send a proactive notice on the orders flagged at the recall-oriented threshold, which covers five

in six of those that will actually miss. Reserve expedited freight for the small high-precision tail, and revisit that policy if expediting falls below about R$12 an order. **Customer experience, at delivery:** work the flagged list before the review window closes — 7.6% of deliveries at the tuned threshold, catching 39.8% of the orders that would draw a 1–2 star review at 57.5% precision, or 15.0% at the cost-optimal threshold for a wider net.

These are not two systems: the order flagged at checkout is the order that arrives late, and the order that arrives late is the one flagged for outreach, so the first gives operations a chance to prevent the cost the second is left measuring.

6 Limitations and Conclusion

The models are fitted on a moving target. Mean lead time falls 5.34 days between training and test, and a model fitted on the slower past is systematically off-level on the faster present — Section 4.5 measures that as a positive bias in every region. Hence R^2 against a test-period variance is negative even for a useful model, which is why MAE leads, and hence the models need refitting on a rolling basis rather than once [6]: a deployment should refit monthly and monitor the gap between predicted and realised lead time as the trigger. Calibration belongs in the same cycle — the probabilities hold on the test window, but that is a two-month-ahead check, and a calibration fitted on one prior drifts as the prior does.

The largest driver of lateness is not observable at prediction time. The late rate swings between 1.4% and 21.4% month to month with Black Friday 2017 and the May 2018 truckers' strike. These are exogenous shocks — carrier capacity, national road blockades — that no order-level feature available at checkout can anticipate. This, not the choice of algorithm, is the binding constraint on Problem 1B, and it is why the ceiling was stated in advance rather than a weak classifier presented as a disappointment.

What the data does not contain, and what rests on assumption. The service-quality literature identifies product expectation gaps, seller communication and returns handling among the drivers of dissatisfaction [9, 12]; none appears in these nine tables, so the Problem 2 residual is unobserved cause rather than noise, and the free-text review columns (present for 11.7% and 41.3% of reviews) are too sparse to recover it. Olist publishes no cost figures, so Section 5.2 states its assumptions and reports break-even precision — the quantity that decides the recommendation and moves with the assumption. And selection votes on three out-of-sample windows covering roughly February to June 2018: the right trade under drift, but a smaller sample than cross-validation uses, and Section 4.2 records a case where the rule preferred a model that does not transfer. We report that rather than quietly selecting on the test window.

Generalisation, and what these scores are not. Everything here is Brazil, 2016–2018, on one marketplace with a distinctive geography: 59.7% of sellers sit in São Paulo state against 42.0% of customers, so most orders cross the country and the median one travels 434 km — the distance-driven structure the models learn is a property of that asymmetry. The outputs are triage scores for internal prioritisation, not entitlements, and must not be used to penalise a seller, price a customer, or drive an automated decision about someone without review. The review-risk model in particular is largely a restatement of how late a parcel was, and a seller penalised by it would in many cases be penalised for a carrier's failure.

Conclusion

Two problems, at two prediction moments, from one dataset and one feature pipeline. The regression reaches MAE 3.90 days — 41% below a constant fitted on the past and 60% below the platform's own promise — and its output, thresholded against that promise, makes a better late-order flag (2.89× the no-skill floor) than any classifier trained directly on the binary label (1.24×), so the brief's dual framing turns out to be the substantively better design. The satisfaction model reaches 3.75× its floor on a target where always predicting "satisfied" scores 89% accuracy and catches nothing, and scores identically on customers it has never seen.

The methodological findings travel further. Cross-validation inside a drifting training period reorders the candidates relative to the forward window, and a two-month validation period is short enough to reorder them again — so selection belongs on several recent out-of-sample windows judged by rank, and even then it can fail. Weighting the loss for class imbalance leaves ranking unchanged and inflates predicted risk threefold, so the threshold is the right lever. Raw PR-AUC is not comparable across periods with different base rates while its lift over the floor is. And the most useful business result is a negative one: a flag with ROC-AUC 0.782 is genuinely informative and still does not justify an expensive intervention, because its precision never clears the break-even the cost ratio demands.

References

[1] Christoph Bergmeir and José M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012. doi: 10.1016/j.ins.2011.12.028.

[2] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(10):281–305, 2012.

[3] Ruomeng Cui, Zhikun Lu, Tianshu Sun, and Joseph M. Golden. Sooner or later? promising delivery speed in online retail. *Manufacturing & Service Operations Management*, 26(1): 233–251, 2024. doi: 10.1287/msom.2021.0174.

[4] Vinayak Deshpande and Pradeep K. Pendem. Logistics performance, ratings, and its impact on customer purchasing behavior and sales in E-Commerce platforms. *Manufacturing & Service Operations Management*, 25(3):827–845, 2023. doi: 10.1287/msom.2021.1045.

[5] Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI-01)*, pages 973–978, Seattle, Washington, 2001.

[6] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 2014. doi: 10.1145/2523813.

[7] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 2012. doi: 10.1145/2382577.2382579.

[8] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML '05)*, pages 625–632. ACM, 2005. doi: 10.1145/1102351.1102430.

[9] A. Parasuraman, Valarie A. Zeithaml, and Arvind Malhotra. E-S-QUAL: A multiple-item scale for assessing electronic service quality. *Journal of Service Research*, 7(3):213–233, 2005. doi: 10.1177/1094670504271156.

[10] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[11] John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In Alexander J. Smola, Peter L. Bartlett, Bernhard Schölkopf, and Dale Schuurmans, editors, *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, Cambridge, Massachusetts, 1999.

[12] Shashank Rao, Thomas J. Goldsby, Stanley E. Griffis, and Deepak Iyengar. Electronic logistics service quality (e-LSQ): Its impact on the customer's purchase satisfaction and retention. *Journal of Business Logistics*, 32(2):167–179, 2011. doi: 10.1111/j.2158-1592.2011.01014.x.

[13] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3): e0118432, 2015. doi: 10.1371/journal.pone.0118432.

[14] Carolin Strobl, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8:25, 2007. doi: 10.1186/1471-2105-8-25.

[15] Ruben van den Goorbergh, Maarten van Smeden, Dirk Timmerman, and Ben Van Calster. The harm of class imbalance corrections for risk prediction models: Illustration and simulation using logistic regression. *Journal of the American Medical Informatics Association*, 29 (9):1525–1534, 2022. doi: 10.1093/jamia/ocac093.

A Reproducing this report

Table 7: Everything in this document is generated. The order below is the order to run it in. The fitting step dominates; the rest takes under two minutes.

Command	What it produces
`pip install -r requirements-dev.txt`	The full analysis environment. `requirements.txt` alone is the deployment runtime
`python tests/test_leakage.py`	32 validation checks: join grain, the history construction against a brute-force recomputation, the leakage guard in both directions, split disjointness, Phase 1 backward compatibility
`python src/train_models.py`	Fits every model, scores the test window once, writes `models/` and every result table into `report/results/`
`python src/figures_phase2.py`	Draws every figure as vector PDF into `report/figures/` from those tables
`python src/phase2_stats.py`	Distils every number quoted in this report into `report/stats_phase2.txt`. Diff it against the manuscript
`python src/report_stats.py`	Still reproduces the *Phase 1* numbers unchanged — the check that the Phase 2 feature additions are backward compatible
`streamlit run dashboard/app.py`	The dashboard, including the Predict tab that serves the models above

`SEED = 42` is set in every estimator, every randomised search and every subsample. Both requirements files pin exact versions on Python $\leq$ 3.12, the interpreter every reported number was produced on. Model artifacts record the `scikit-learn` version that wrote them and `modeling.load_model()` checks it on the way back in.

B Success criteria registered in advance

Written into `config.EXPECTED_BANDS` before any Phase 2 model was tuned. `evaluate.check_bands()` compares the headline results against them at the end of every training run and writes its verdicts to `report/results/bands.csv`. The alarm is direction-aware: for an error metric the suspicious end of the interval is the low one, for a score the high one, and confusing the two would point the check at the shortfalls and wave the leaks through.

Problem	Metric	Registered band	Where the range came from
1A	MAE (days)	3.3 – 4.3	Phase 1 reported 3.89 untuned
1B	balanced accuracy	0.58 – 0.72	Phase 1 reported 0.58 for a direct classifier
1B	ROC-AUC	0.72 – 0.85	Untuned scoping probe reached 0.79 for the derived route
2	balanced accuracy	0.63 – 0.74	Phase 1 reported 0.69
2	ROC-AUC	0.70 – 0.79	Phase 1 reported 0.73
2	PR-AUC	0.34 – 0.48	Phase 1 reported 0.40 against a 0.11 floor

C Hyperparameter search spaces and selected values

Family	Rung	Classification	Regression
A — Linear	0	`LogisticRegression`	`LinearRegression`
	1	`LogisticRegression` (ℓ_2, tuned C)	`Ridge` (tuned α)
B — Tree-based	0	`DecisionTreeClassifier`	`DecisionTreeRegressor`
	1	`RandomForestClassifier`	`RandomForestRegressor`
	2	`HistGradientBoostingClassifier`	`HistGradientBoostingRegressor`
C — Ensemble	0	`VotingClassifier` (soft, A+B)	`VotingRegressor` (A+B)
	1	`StackingClassifier` (A+B)	`StackingRegressor` (A+B)

Every tuned model was searched with `RandomizedSearchCV` over the distributions below, scored by five-fold expanding-window cross-validation inside the training period. Thirty draws per model, except the random forests, which get six: a forest costs one to two orders of magnitude more per fit than the other rungs, and Phase 1 had already shown the forest and the boosted trees landing within 0.01 days of each other. That is a documented trade rather than a hidden one.

Table 8: Search spaces. Selected values are in `report/results/reg_p1a.csv`, `clf_p1b.csv` and `clf_p2.csv`, and are reproduced in `models/model_metadata.json`.

Model	Draws	Distributions
`LinearRegression` / `LogisticRegression`	—	No free parameters. Fitted and scored under the same protocol so the row is comparable
`Ridge`	30	$\alpha \sim \text{loguniform}(10^{-3}, 10^{3})$
`LogisticRegression` (ℓ_2)	30	$C \sim \text{loguniform}(10^{-3}, 10^{2})$
`DecisionTree`	30	`max_depth` $\sim$ randint(3, 20); `min_samples_leaf` $\sim$ randint(5, 200) [classifier: (20, 400)]; `max_features` $\sim U(0.4, 1.0)$
`RandomForest`	6	150 trees; `max_depth` $\sim$ randint(8, 18) [classifier: (6, 16)]; `min_samples_leaf` $\sim$ randint(20, 140) [classifier: (30, 180)]; `max_features` $\sim U(0.15, 0.40)$
`HistGradientBoosting`	30	`max_iter` $\sim$ randint(120, 500); `learning_rate` $\sim$ loguniform$(0.02, 0.3)$; `max_leaf_nodes` $\sim$ randint(15, 64); `min_samples_leaf` $\sim$ randint(10, 120); `l2_regularization` $\sim$ loguniform$(10^{-4}, 10)$
`Voting`	—	Weights fixed at 1:2 in the tree's favour. A stated prior, not a tuned parameter: under drift the linear member is materially weaker and an equal blend would be worse than its better half
`Stacking`	—	Meta-learner `Ridge`($\alpha = 1$) / `LogisticRegression`; inner CV five contiguous folds — see the note below

A compromise inside stacking, stated plainly. `StackingClassifier` and `StackingRegressor` build their meta-features with `cross_val_predict`, which requires the inner folds to *partition* the training set: every row must be predicted exactly once. An expanding window partitions nothing — the earliest rows are never in a test fold — so the closest valid approximation is unshuffled `KFold`, which at least keeps each fold contiguous in time. The meta-learner is consequently fitted on out-of-fold predictions partly made by models trained on later orders.

The residual optimism is confined to the meta-learner's coefficients; the base models are refitted on the whole training period afterwards, and the outer protocol scoring the finished ensemble is still the expanding window. It matters less than it otherwise would because of how the result lands: the ensemble is *flattered* by this arrangement and still does not win, which makes the conclusion in Section 4.5 safer rather than weaker.

D Re-checking a Phase 1 claim about least squares

Phase 1 reported that ordinary least squares was unusable on this design matrix — "rank-deficient, so its coefficients explode". Phase 2 re-ran it four ways. The observation reproduces exactly; the conclusion drawn from it needed correcting.

Table 9: Reproducible via `report/results/encoding_diagnostic.csv`. The deficiency is exactly four — one per categorical, because each one-hot block sums to one and is therefore collinear with the intercept.

Encoding	rank / cols	deficiency	max $\|\beta\|$	CV MAE	test MAE
OLS, full one-hot block	101 / 105	4	2.7×10^{13}	3.9×10^{11}	4.181
OLS, one level dropped	101 / 101	0	14.7	5.242	4.181
Ridge, full one-hot block	101 / 105	4	9.82	5.241	4.182
Ridge, one level dropped	101 / 101	0	12.5	5.240	4.183

Least squares still returns a minimum-norm solution and its *predictions* are fine — 4.181 days on the test window, indistinguishable from Ridge. What is not fine is the coefficient vector at 10^{13}, and the cancellation holding it together is fragile enough to fail on one of five expanding-window folds, which is why the cross-validated error is astronomical while the single-split test error looks healthy. A single train/test split would never have shown it.

Dropping one level per categorical restores full rank, so the linear family is encoded that way. The compromise is small and worth stating: an all-zero row is also how the encoder represents a level it has never seen, so under `drop="first"` a novel payment type is scored as the reference level. One payment level is unseen in the test window, affecting a handful of orders. Leaving the design rank-deficient costs more.

The last row carries the real lesson, and it is not the one Phase 1 drew. Once the design is full rank, Ridge and OLS agree to three decimals. Regularisation buys numerical stability here, not accuracy.

E Full result tables

The body reports the models that matter. The complete tables — every model, every metric, cross-validated, validation and test scores side by side — are written by `src/train_models.py` to `report/results/`:

File	Contents
`reg_p1a.csv`	Problem 1A: baselines, both ladders, ensembles; MAE, RMSE, R^2
`clf_p1b.csv`	Problem 1B: five direct classifiers and two derived routes
`clf_p2.csv`	Problem 2: both ladders and both ensembles, all metrics
`encoding_diagnostic.csv`	The Phase 1 re-check of Table 9
`cv_protocol.csv`	Temporal against shuffled CV against the forward test
`weight_ablation.csv`	Class weighting: discrimination and calibration
`calibration_p2.csv`, `threshold_sweep_p2.csv`	Reliability and the operating-point sweep
`cost_p2.csv`, `cost_p1b.csv`	Expected net benefit by threshold
`perm_p1a.csv`, `perm_p2.csv`	Permutation importance on the test window
`segments_*.csv`	Performance by region, category and test month
`predictions_*.csv`	Per-order test predictions, from which the figures are drawn
`bands.csv`	Headline metrics against the bands registered in advance

F Deployment

The dashboard at `https://it5006-group-project-10.streamlit.app` carries the five Phase 1 exploratory tabs unchanged and adds two.

Predict scores a single order entered as it would look at checkout — route, basket, timing and payment — and returns the expected lead time, the risk of missing the promised date against the tuned threshold, and the resulting low-review risk. It also scores an uploaded CSV and returns it with the three predictions appended. Nothing there reimplements the training-time feature engineering: a form submission is turned back into a one-row order frame and pushed through the same `features.py` builders the models were fitted on, which is what stops a feature being computed one way in training and another way in production. The one thing a form cannot supply is a seller's track record, so `models/history_lookup.csv` carries those aggregates at each level of the same fallback chain `add_history()` uses.

Model card reports the training window, the selected models and their hyperparameters, the test metrics, the decision thresholds, and an explicit list of situations in which the models should not be trusted.

Engineering constraints are recorded rather than discovered later. The deployed estimators are the histogram-boosted variants, whose pickles are a few hundred kilobytes; a random forest of the same accuracy pickles to 24–30 MB, which is hostile to a shared container. Total artifact size is under 2 MB. `requirements.txt` pins `scikit-learn` to the exact version the models were trained on and the loader raises a visible warning if it ever drifts. Models are loaded lazily, so a visitor who only browses the exploratory tabs never pays for unpickling one. Invalid input — a promised date before the purchase date, a negative price, more sellers than items, a malformed or empty upload, a missing column — produces a specific message rather than a traceback. No credentials appear anywhere in the repository.

G Contributions

Member	Contribution
Pan Pinyou	Problem scoping and stakeholder framing; feature engineering for the track-record features; business interpretation and the cost analysis; report Sections 1 and 5
Li Yudan	Modelling pipeline and the family ladders; hyperparameter search; imbalance and calibration analysis; report Sections 3 and 4.3
Zhu Gengyu	Data pipeline and the leakage guard; cross-validation protocol and the direct-against-derived comparison; dashboard, deployment and reproducibility infrastructure; report Sections 2, 4.1 and 6

All three members reviewed every section and verified the reported numbers against `report/stats_phase2.txt`.

H Figures

Every figure below is drawn by `src/figures_phase2.py` from the same CSV tables the body's numbers come from, so a figure cannot disagree with a table.

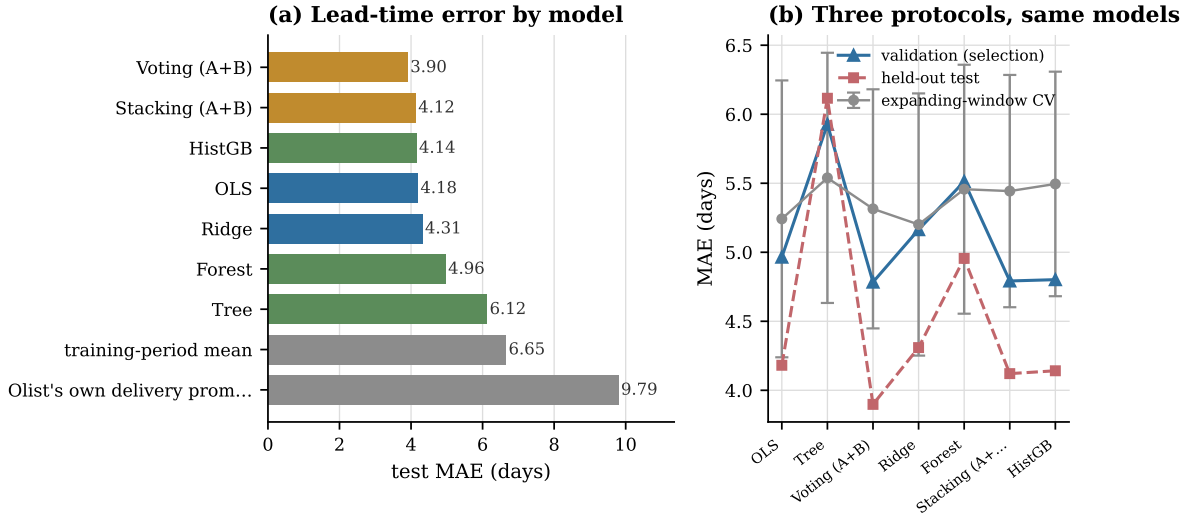


Figure 1: Lead-time error by model, and the same models under three protocols. Panel (b) is the argument of Section 3.2: expanding-window cross-validation does not order the candidates the way the validation period and the forward window — which agree with each other — do.

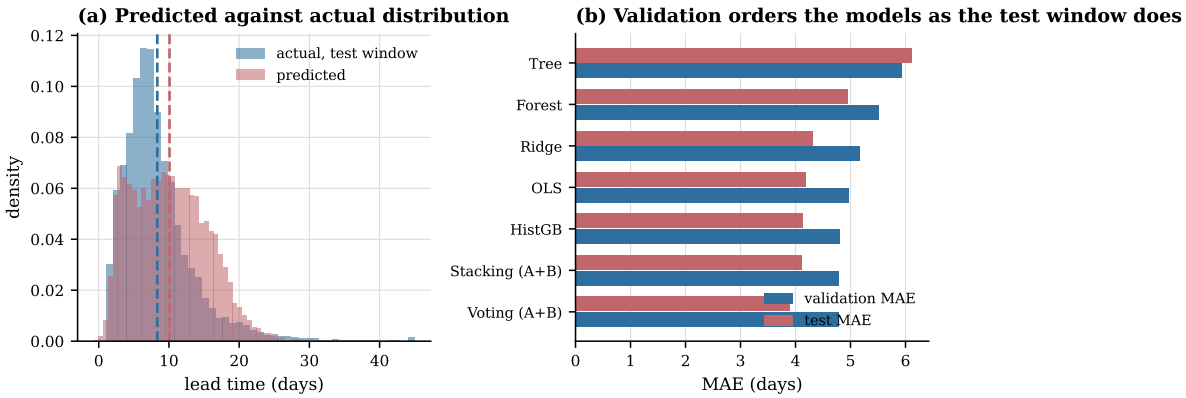


Figure 2: Why blending a linear model with a boosted one helps under drift. A tree cannot predict outside the range of the targets it was fitted on; lead times in the test window are systematically shorter than anything common in training. Panel (b) shows the validation period ordering the models as the test window does.

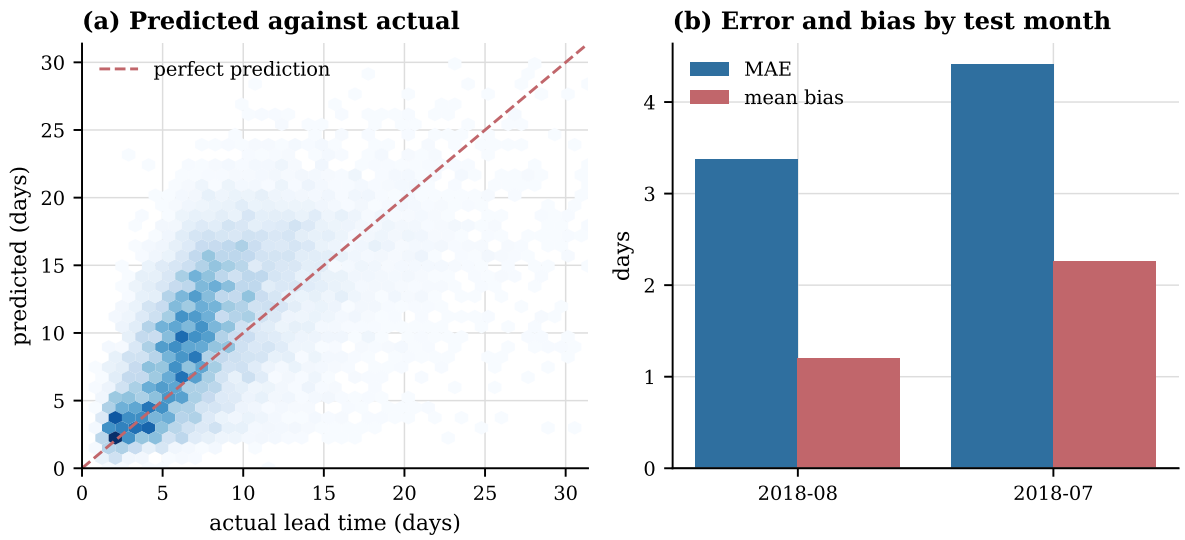


Figure 3: Where the regression is wrong. The bias in panel (b) is positive in both test months: a model fitted on a slower past over-predicts how long a parcel will take.

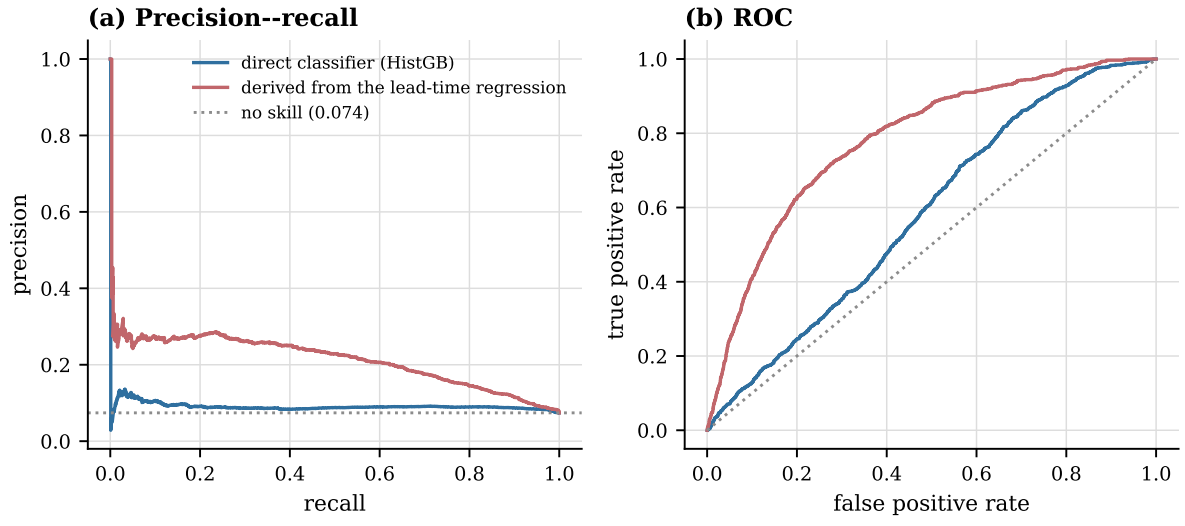


Figure 4: Problem 1B. The flag derived from the lead-time regression against the best purpose-built classifier, on the forward test window.

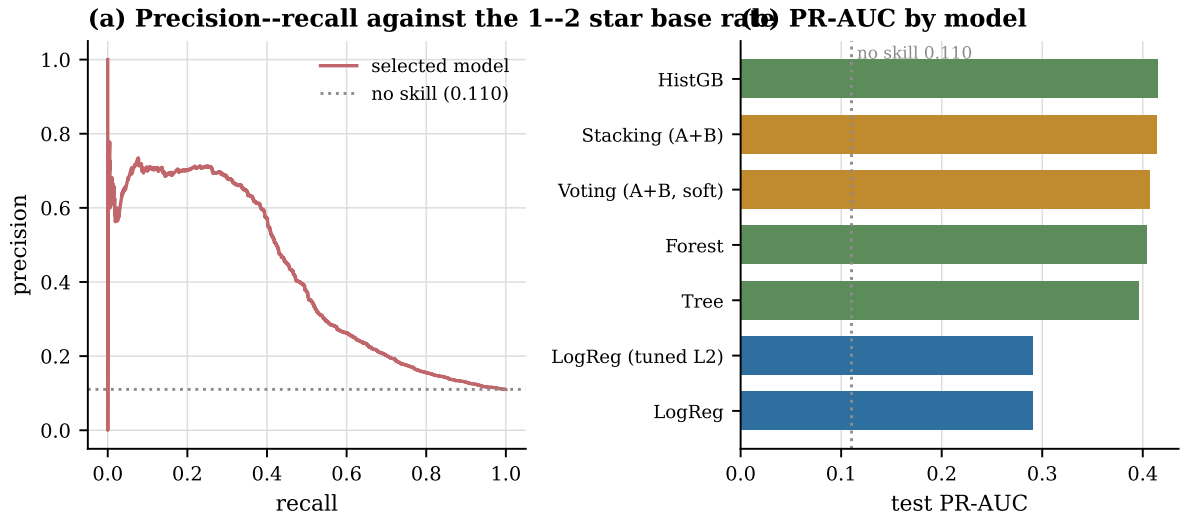


Figure 5: Problem 2 against its no-skill floor, and PR-AUC by model. Most of the achievable signal is already available from a single depth-limited tree.

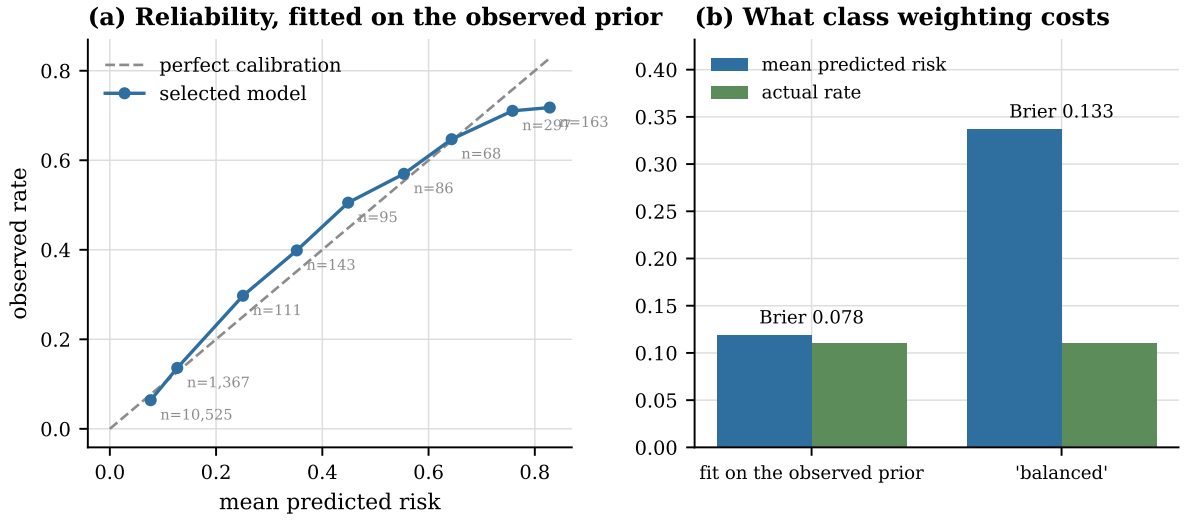


Figure 6: Calibration, and what class weighting costs. Weighting the loss leaves ranking unchanged and inflates mean predicted risk threefold against the actual rate.

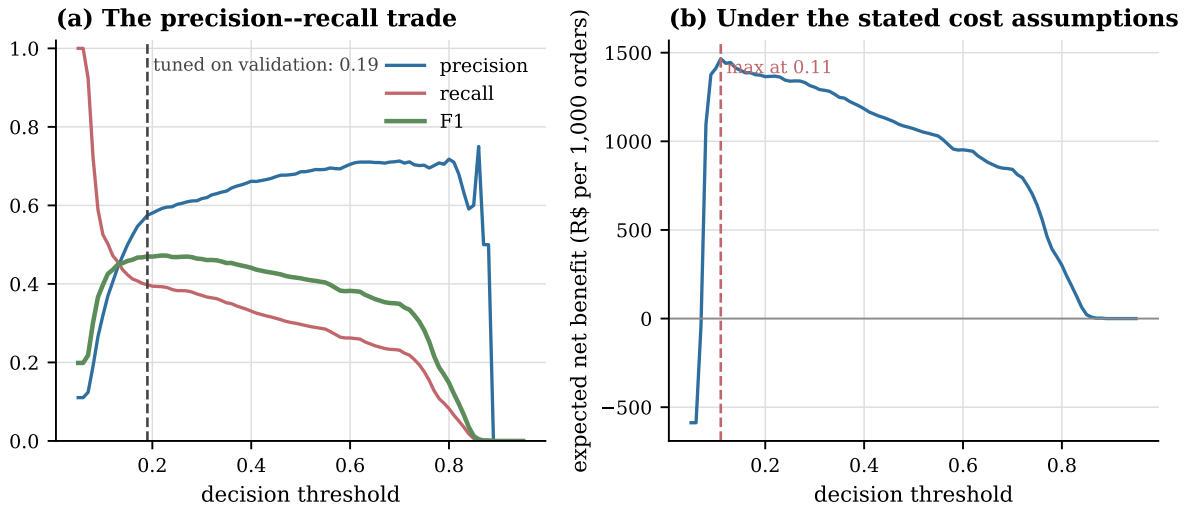


Figure 7: Choosing the operating point for the review model, and what it is worth under the stated cost assumptions.

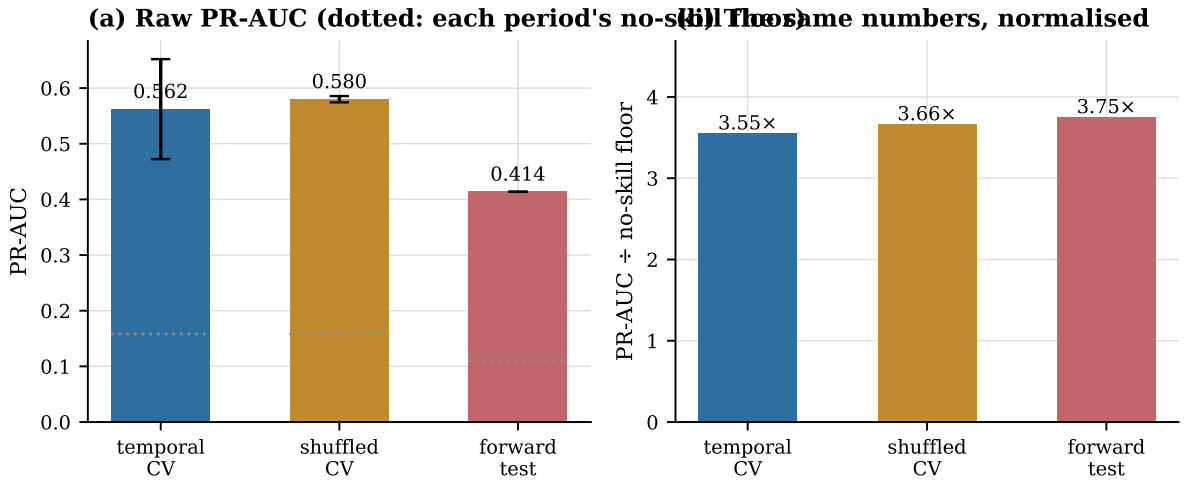


Figure 8: One model, three protocols. Raw average precision disagrees across periods; divided by each period's no-skill floor the three agree to within 0.2 of a multiple.

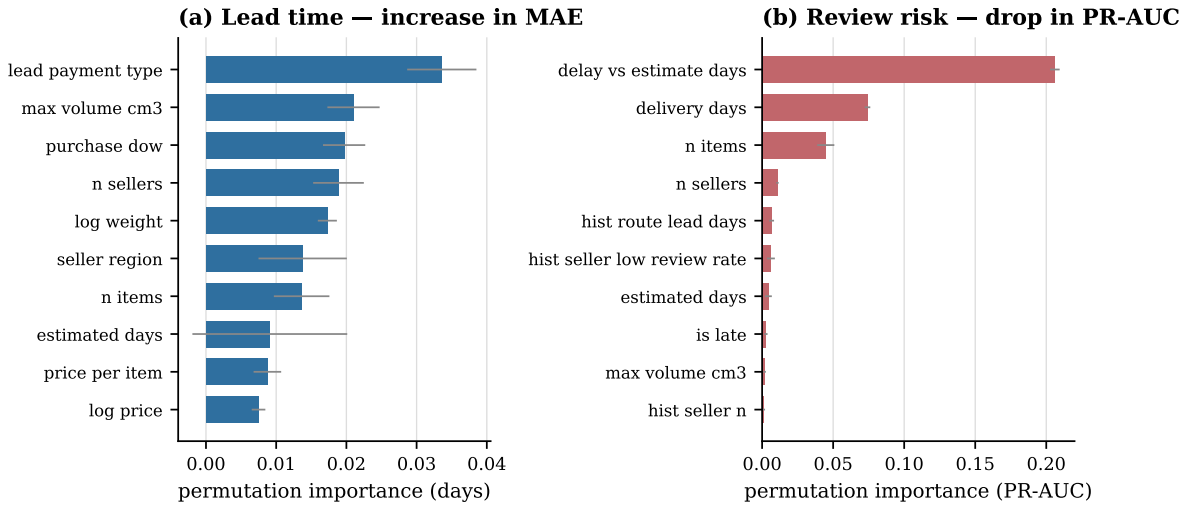


Figure 9: Permutation importance on the test window. The review model rests on two features; the lead-time model on none in particular, which is a property of correlated predictors as much as of the model.

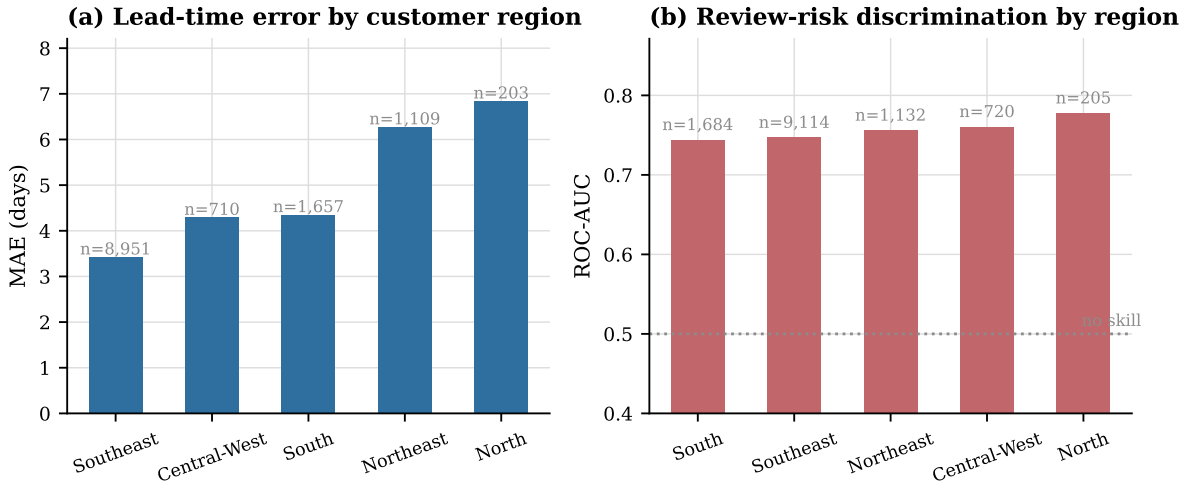


Figure 10: Performance by customer region. The review model is stable across all five; the regression is not, and degrades exactly where deliveries are longest.

AI Tool Declaration

We used OpenAI's ChatGPT and Anthropic's Claude to generate and validate ideas, improve expression and refine our writing. We also used the models to debug our code. All analytical decisions, results and conclusions were reviewed and verified by the team against the underlying data, and we are responsible for the content and quality of the submitted work.